TEAM VIFY AI / GUIA DE IMPLEMENTACIÓN

De idea a demo en 20 horas

Guía de programación de Agua Clara AI

PANAMÁ OESTE · COORDINACIÓN DE CISTERNAS

EQUIPO

Frida Leis · Yohany Kavacila · Victor Montes

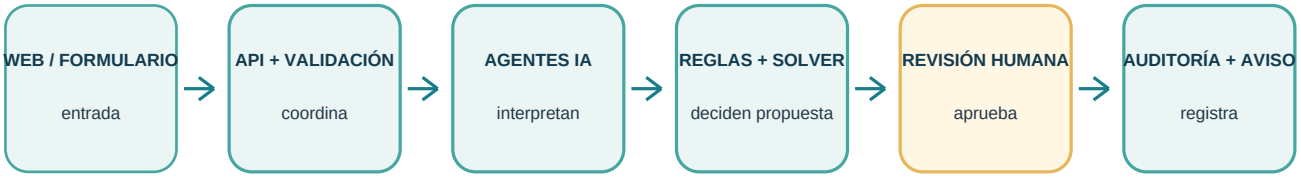
00 | Antes de escribir código

Meta de la demo: un operador ingresa reportes, ve cuáles son prioritarios, obtiene una propuesta de asignación, la revisa y registra su decisión. Construyan el camino completo con datos ficticios antes de añadir canales externos o hardware.

REGLA PARA NO PERDER TIEMPO

Cada módulo debe tener una entrada clara, una salida clara y una forma sencilla de probarse. Si algo no se puede demostrar de punta a punta, se deja como extensión.

Arquitectura de la demo



El modelo interpreta; reglas deterministas calculan; una persona autoriza.

Orden de construcción

Primero	Después	Al final
Datos y reglas de prioridad	Interfaz y API	Mapa, WhatsApp, audio y ESP32
Un reporte puede recorrer el sistema completo	El operador puede corregir y aprobar	Pulir visuales y extras opcionales

01 | Preparar el proyecto

Empiecen con una aplicación Python pequeña. Una persona crea el repositorio y el esqueleto; las demás implementan módulos separados con contratos acordados. Eviten que dos integrantes editen el mismo archivo a la vez.

Estructura sugerida

```
agua-clara-ai/
■■■ app.py # interfaz de demostración
■■■ api.py # endpoints y coordinación
■■■ schemas.py # modelos Pydantic compartidos
■■■ agents/
■ ■■■ intake.py
■ ■■■ geolocation.py
■ ■■■ prioritization.py
■ ■■■ planner.py
■ ■■■ communication.py
■ ■■■ supervisor.py
■■■ services/
■ ■■■ storage.py
■ ■■■ audit.py
■ ■■■ demo_data.py
■■■ data/
■ ■■■ sectors.json
■■■ README.md
```

Contrato común: qué recibe cada módulo

Objeto	Campos mínimos
Report	id, created_at, text, channel, sector_hint, hours_without_water, affected_people, severity, evidence, confidence, status
Sector	id, name, district, population, vulnerability, essential_services, latitude, longitude
Tanker	id, capacity_liters, available_at, current_location, status
Assignment	sector_id, tanker_id, liters, stop_order, eta_minutes, reasons, constraints_ok

Arranque técnico

Crear entorno virtual, instalar FastAPI, Uvicorn, Pydantic y la librería de interfaz elegida. Mantener un archivo requirements.txt y datos de prueba incluidos en el repositorio. Si una dependencia da problemas tras 15 minutos, continuar con una alternativa simple y anotarlo.

02 | Orden de programación

Construyan el sistema en este orden. El primer hito funcional debe ocurrir pronto: escribir un caso de prueba manual y verlo transformarse en un reporte guardado.

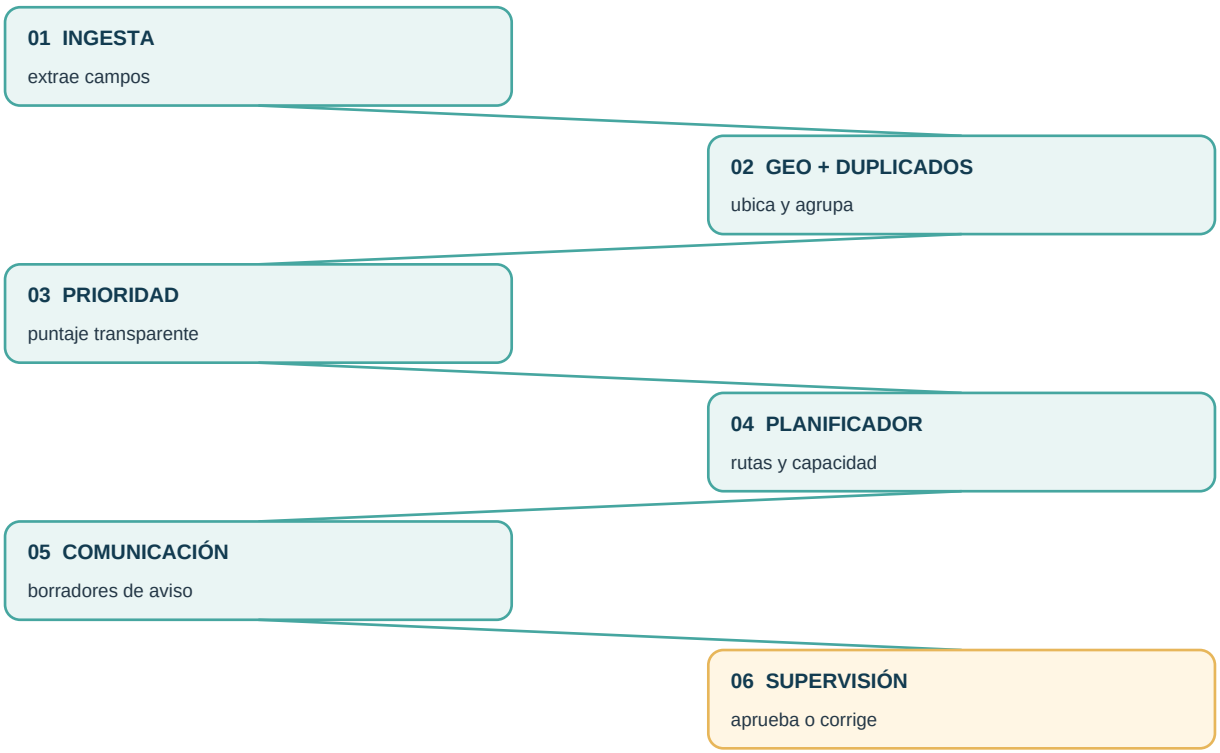
Paso	Implementar	Se considera terminado cuando…
1. Datos compartidos	schemas.py con Report, Sector, Tanker, Assignment y estado del caso.	Los objetos se validan y rechazan datos inválidos.
2. Datos demo	sectors.json y demo_data.py con sectores, cisternas y 15 casos.	La demo carga siempre el mismo conjunto limpio.
3. Guardado y auditoría	storage.py y audit.py con SQLite o JSON append-only.	Cada cambio guarda quién, cuándo y qué cambió.
4. Prioridad	prioritization.py: función pura que devuelve puntuación, rango y razones.	Mismos datos producen el mismo orden y la explicación enumera factores.
5. Ingesta	intake.py: convertir texto en Report con campos faltantes explícitos.	La salida se valida y mantiene texto original; incertidumbre se marca.
6. Geografía y duplicados	geolocation.py: catálogo local y similitud simple texto/sector/tiempo.	Sugiere coincidencias; nunca combina automáticamente casos dudosos.
7. Planificador	planner.py: asignación factible por capacidad, disponibilidad y ventanas.	Ninguna cisterna excede litros; el operador entiende ruta y razón.
8. Supervisor y aviso	supervisor.py: aprobar, corregir, rechazar; communication.py redacta aviso.	No sale aviso sin aprobación; el cambio queda en auditoría.

CRITERIO DE FIN POR MÓDULO

El módulo importa, recibe un objeto de ejemplo, devuelve el tipo acordado y maneja al menos un dato faltante sin romper toda la aplicación.

03 | Los seis agentes y sus conexiones

Trátenlos como funciones/herramientas con nombres claros. No hace falta que seis modelos conversen entre sí. El orquestador llama cada etapa en orden y conserva el estado del caso.



Todos producen y consumen datos estructurados; la aprobación humana cierra el ciclo.

Qué programa cada agente

Agente	Función de entrada / salida	No debe hacer
Ingesta	texto/audio transcrito/imagen descrita → Report + campos inciertos	Inventar información faltante.
Geolocalización y deduplicación	Report + catálogo → sector sugerido + posibles duplicados	Fusionar casos ambiguos sin confirmación.
Priorización	Reportes y sectores confirmados → score + factores + orden	Usar popularidad o volumen de quejas como prioridad.
Planificador	sectores ordenados + cisternas → Assignment[] + restricciones	Superar capacidad o asegurar ETA sin datos.
Comunicación	estado aprobado → borrador claro en texto/audio opcional	Enviar sin aprobación o prometer recursos.
Supervisión humana	plan + razones → aprobado/editado/rechazado + responsable	Delegar la decisión final al modelo.

04 | La lógica que no debe quedar en manos del modelo

El modelo de IA puede extraer información de un mensaje libre y devolver JSON validado. El código normal debe calcular puntajes, validar restricciones, guardar decisiones y decidir si el aviso puede publicarse.

Puntaje inicial explicable

Para el prototipo, normalicen cada factor a 0–1 y usen pesos definidos por el equipo y visibles en la interfaz. Por ejemplo: horas sin agua 40%, población afectada 30%, vulnerabilidad 30%. Documenten que son parámetros de demostración, no política oficial.

```
score = 0.40 × horas_normalizadas
+ 0.30 × población_normalizada
+ 0.30 × vulnerabilidad
```

Restricciones mínimas del plan

- litros_asignados ≤ capacidad_litros de cada cisterna.
- Una cisterna disponible se asigna a un solo recorrido simultáneo.
- No planificar antes de hora_disponible ni fuera de una ventana confirmada.
- Si no alcanza la oferta, mostrar sectores pendientes y motivo.
- Mostrar puntaje y factores; desempatar de forma determinista.

Pseudoflujo de orquestación

```
report = intake.parse(raw_input)
report = geolocation.resolve(report, sector_catalog)
matches = geolocation.find_duplicates(report, existing_reports)
await human.confirm(report, matches)
scores = prioritization.rank(confirmed_reports, sectors)
plan = planner.build(scores, available_tankers)
decision = await supervisor.review(plan)
audit.append(decision)
if decision.approved: communication.draft(decision)
```

PRINCIPIO

La IA presenta evidencia estructurada. Las reglas hacen cálculos repetibles. La persona toma la decisión operativa.

05 | Conectar API e interfaz

Mantengan una sola ruta feliz en la demo: cargar → revisar → planificar → aprobar. La API coordina los módulos; la interfaz no replica las reglas del servidor.

Endpoints mínimos si usan FastAPI

Ruta	Acción
POST /reports	Valida y crea un reporte; devuelve campos extraídos y confianza.
GET /reports	Lista reportes y estado.
POST /reports/{id}/confirm	Guarda correcciones del operador.
POST /plan	Calcula prioridades y propuesta logística.
POST /plan/{id}/decision	Aprueba, rechaza o modifica; exige usuario y motivo.
GET /audit	Muestra eventos recientes de auditoría.

Pantallas mínimas

Vista	Elementos que debe mostrar
Bandeja	Reportes, sector, horas sin agua, severidad, confianza y estado.
Detalle	Texto original, campos extraídos, campos faltantes y duplicados sugeridos.
Propuesta	Orden por prioridad, score y factores, cisternas, capacidad y pendientes.
Aprobación	Botones Aprobar / Editar / Rechazar; nombre del operador y motivo.
Estado público demo	Mensaje borrador con estado confirmado, sin datos personales.

Datos demo

Usen nombres de sectores ficticios o datos públicos preparados. Incluyan variedad de horas sin agua, poblaciones, vulnerabilidades, servicio esencial, reportes duplicados, ubicación ambigua y falta de cisternas.

06 | Probar antes de agregar extras

La evaluación no es una etapa final solamente: úsela para decidir si cada módulo está listo. Mantengan los 15 casos en un archivo y repítanlos al integrar cambios.

Casos	Cubrir	Resultado esperado
1–4	Extracción de campos con faltantes y texto ambiguo.	Campos correctos o marcados como inciertos.
5–7	Mismo incidente repetido, texto parecido pero caso distinto, ubicación ambigua.	Sugerencias revisables; sin fusión equivocada.
8–11	Sectores distintos en horas, población, vulnerabilidad y servicio esencial.	Orden coincide con fórmula acordada; razones visibles.
12–13	Capacidad suficiente e insuficiente.	Nunca exceder capacidad; mostrar pendientes.
14–15	Comunicación y control humano.	Aviso no se publica sin aprobación; queda registro.

Lista de aceptación del MVP

- La app corre en el portátil del equipo desde instrucciones del README.
- Un reporte nuevo se guarda y aparece en bandeja.
- El operador puede corregir sector y campos detectados.
- Duplicados se sugieren con contexto, no se borran.
- Prioridad y plan son deterministas y explicables.
- La capacidad de cisternas nunca se excede.
- Aprobar, editar o rechazar registra responsable y motivo.
- Ninguna notificación externa se envía automáticamente.
- Los 15 casos se recorren y se muestra un resumen de resultados.

Respaldo por si falla la IA

Cargar directamente un Report de ejemplo desde formulario. La app debe seguir funcionando con la API de IA desconectada. Así la demostración no depende de internet ni de una clave de modelo.

07 | Reparto del trabajo y 20 horas

Dividan por módulos y acuerden contratos compartidos antes de paralelizar. Una persona mantiene integración y demo de extremo a extremo; cada quien entrega cambios pequeños que el equipo pueda ejecutar.

Bloque	Trabajo conjunto	Hito
0–2 h	Acordar schemas, pesos, datos demo, estructura y roles. Crear app vacía y correrla.	Esqueleto compartido funciona.
2–6 h	Construir bandeja, almacenamiento, datos de sectores y auditoría.	Reporte manual aparece guardado.
6–10 h	Implementar extracción estructurada y catálogo/geolocalización; preparar sugerencias de duplicados.	Reporte puede confirmarse en la UI.
10–14 h	Puntaje determinista y asignación factible con cisternas de demo.	Propuesta muestra razones y capacidad.
14–17 h	Aprobación/edición, historial y borrador de comunicación.	Ciclo operativo completo y auditable.
17–20 h	Ejecutar 15 casos, corregir fallos críticos, ensayar pitch y tener respaldo offline.	Demo repetible de 3 minutos.

Roles de equipo sugeridos

Persona	Responsabilidad inicial	Apoya en
Frida Leis	Interfaz: bandeja, detalle y revisión humana.	Pruebas de usabilidad y guion.
Yohany Kavacila	Datos, ingesta, catálogo y manejo de duplicados.	Casos de evaluación y validación.
Victor Montes	API, priorización, planificación y auditoría.	Integración y ejecución demo.

Todos revisan datos, conectan módulos y ensayan la demo. Redistribuyan según fortalezas reales del equipo.

08 | Cómo empezar y cómo terminar

Primeros 30 minutos

1. Abran un repositorio y creen la estructura acordada.
2. Elijan quién integra y definan el objeto Report en schemas.py.
3. Escriban juntos cinco sectores y tres cisternas de ejemplo.
4. Hagan que app.py arranque y muestre “Agua Clara AI lista”.
5. Ingresen un reporte de texto manual y confirmen que se guarda.
6. Anoten el primer hito como completado; luego implementen los módulos en el orden de esta guía.

Congelen funciones nuevas cuando queden 3 horas

En ese momento, no agreguen canales ni hardware. Repasen la lista de aceptación, arreglen errores de recorrido y preparen un plan B con datos precargados y capturas o grabación local.

Guion de demo en 3 minutos

Tiempo	Mostrar
0:00–0:30	El problema: varios reportes, mismos incidentes, necesidades distintas.
0:30–1:00	Ingresar un reporte y confirmar qué entendió el sistema; enseñar incertidumbre.
1:00–1:45	Abrir duplicados sugeridos y confirmar ubicación; mostrar reglas de prioridad.
1:45–2:30	Generar el plan, verificar capacidad y explicar por qué un sector va primero.
2:30–3:00	Editar/aprobar; mostrar auditoría y el aviso preparado, todavía bajo control humano.

Definición de terminado

LA DEMO ESTÁ LISTA CUANDO

Desde una pantalla limpia se puede recorrer un reporte hasta una decisión aprobada; cada etapa deja evidencia legible; el plan respeta capacidad; los casos de prueba no se rompen y el pitch explica límites y siguiente paso.

TEAM VIFY AI · Agua Clara AI